AAC on Existing AAP — Production-Oriented Install Guide

Table of Contents

1. What you get . . . 1
2. Architecture recommendation . . . 2
3. Prerequisites . . . 2
4. Process . . . 2
5. AAC Host setup . . . 2
   5.1. Basic requirements . . . 2
   5.2. Repository checkout . . . 3
6. Secrets setup . . . 3
   6.1. CAA signing key (required for CIS playbooks that enable CAA) . . . 4
   6.2. Encrypt vault_secrets.yml . . . 4
   6.3. Container secrets (podman) . . . 4
7. OPA installation . . . 5
   7.1. Prerequisites . . . 5
   7.2. OPA Containers installation . . . 5
   7.3. PostgreSQL installation . . . 5
   7.4. REGO policy loading . . . 6
   7.5. Grafana installation . . . 7
   7.6. Firewall configuration . . . 8
   7.7. Quadlets . . . 8
8. AAP Integration . . . 9
   8.1. Project Sync . . . 9
   8.2. Build and push the Compliance EE . . . 9
   8.3. Launch the Compliance assessment . . . 10

Estimated time: 2 to 4 hours for a careful first install.

1. What you get

- OPA containers (`opa-security`, `opa-compliance`, `opa-ot`)
- REGO policies loaded from the `policies/` submodule
- A **dedicated** AAC PostgreSQL database
- Grafana dashboards
- An AAP project + job templates

- An Execution Environment

2. Architecture recommendation

Prefer a dedicated AAC host to provide clear blast radius, capacity, and naming. Do not risk AAP data or container names EDA and MCP are optional for a first production cut.

3. Prerequisites

- AAP **2.5.5** or later
- RHEL for `aac_host`
- Outbound HTTPS to GitHub, console.redhat.com, and registry.redhat.io
- Firewall capacity for AAC ports

4. Process

1. Set up `aac_host`
2. Repository checkout
3. Secrets setup
4. OPA installation
5. PostgreSQL installation
6. REGO policy loading
7. Grafana installation
8. AAP project sync and Galaxy credentials
9. AAC Compliance EE installation
10. Verification checklist
11. AAC Compliance assessment

5. AAC Host setup

5.1. Basic requirements

RHEL server with `compliance` user, registered to Red Hat.

```
ssh compliance@compliance.home.lab
echo "compliance ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/compliance
sudo dnf install -y podman ansible-core vim tree jq curl pip bash-completion
sudo dnf upgrade -y && sudo systemctl reboot
mkdir ~/repos/
```

5.2. Repository checkout

```
cd ~/repos/
# Pick one — HTTPS or SSH:
# git clone https://github.com/R0tt3nT0m4t0/compliance.git
git clone git@github.com:R0tt3nT0m4t0/compliance.git
cd compliance
git submodule update --init --recursive
tree -L 2 policies/benchmarks/
```

You might have to generate and register SSH key with GitHub.

```
ssh-keygen -t ed25519 -C "compliance@home.lab"
cat ~/.ssh/id_ed25519.pub
```

Add the public key to GitHub.

6. Secrets setup

Make a copy of the example secrets file and edit the PLACEHOLDERs. Never commit real secrets.

```
cp ansible/vars/vault_secrets.yml.example ansible/vars/vault_secrets.yml
```

NOTE	Credentials are **not** inherited by all templates in a project — attach them per job template.

```
vim ansible/vars/vault_secrets.yml
```

```
---
# CATEGORY 1 — MUST PROVIDE (personal credentials — unique per installer)
vault_redhat_username: some_username
vault_redhat_offline_token: eyJhbGciOiJIUzUxMiIsInR5cCIgOiAiSld...zgFe_qHOt
vault_github_pat: ghp_...KA
vault_quay_username: some_username
vault_quay_password: some_password

# CATEGORY 4 — MUST SET
vault_aap_admin_password: redhat123

# CATEGORY 5 — CAN USE DEFAULTS
compliance_db_password: compliance
compliance_reader_password: compliance
grafana_admin_password: GrafanaAAC2026
```

```
# CATEGORY 6 — CAA
caa_signing_private_key: |
  -----BEGIN ENCRYPTED SIGSTORE PRIVATE KEY-----
  eyJrZGYiOnsib...
  aFQ4bUFIWld2Y0lsVHE2aFM5b09QTlJ0V2c9PSJ9
  -----END ENCRYPTED SIGSTORE PRIVATE KEY-----
caa_signing_password: ""
```

6.1. CAA signing key (required for CIS playbooks that enable CAA)

Generate keys using the cosign binary from a container.

```
cid=$(podman create registry.access.redhat.com/hi/cosign:3)
podman cp "$cid":/usr/bin/cosign ./cosign
podman rm "$cid"
chmod +x ./cosign
COSIGN_PASSWORD="" ./cosign generate-key-pair
```

IMPORTANT	The **public** key is for auditors (`cosign verify-blob`); do **not** inject it as the signing secret on the template.

6.2. Encrypt vault_secrets.yml

```
ansible-vault encrypt ansible/vars/vault_secrets.yml
```

6.3. Container secrets (podman)

Create podman secrets so passwords never appear in `podman run` commands or process listings.

```
podman secret list
printf 'compliance' | podman secret create compliance_db_password -
printf 'compliance' | podman secret create compliance_reader_password -
printf 'GrafanaAAC2026' | podman secret create grafana_admin_password -
podman secret list
```

To rotate a secret later:

```
podman secret rm compliance_db_password
printf 'new_password' | podman secret create compliance_db_password -
```

7. OPA installation

7.1. Prerequisites

First, lets create the `/etc/opa/compliance` and `/etc/opa` directories and set the correct permissions.

```
sudo mkdir -p /etc/opa/compliance
sudo mkdir -p /etc/opa
sudo chown -R compliance:compliance /etc/opa/compliance
sudo chown -R compliance:compliance /etc/opa
```

7.2. OPA Containers installation

Now run the installation playbook

```
ansible-playbook -J ansible/playbooks/deploy_opa_containers.yml \
  -e "aac_host=compliance.home.lab" -e "become_user=root" \
  -e "become=true" -e "become_method=sudo"
```

Verify:

```
curl -s http://compliance.home.lab:8181/health
curl -s http://compliance.home.lab:8182/health
curl -s http://compliance.home.lab:8183/health
# Expect HTTP 200 / JSON health
```

7.3. PostgreSQL installation

```
podman volume create compliance_pgdata

podman run -d --name aac-postgresql \
  -p 5432:5432 \
  -e POSTGRESQL_USER=compliance \
  -e POSTGRESQL_DATABASE=compliance \
  --secret compliance_db_password,type=env,target=POSTGRESQL_PASSWORD \
  -v compliance_pgdata:/var/lib/pgsql/data \
  --restart=always \
  registry.redhat.io/rhel9/postgresql-15:latest
```

Now initialize the database without using the EE container.

Install the dependencies

```
pip3 install --user psycopg2-binary
ansible-galaxy collection install community.postgresql
```

```
ansible-playbook -J ansible/playbooks/postgres_database_init.yml \
  -e "aac_host=compliance.home.lab" \
  -e "pg_host=compliance.home.lab" \
  -e "pg_user=compliance" \
  -e "aac_db_container_name=aac-postgresql" \
  -e "aac_db_skip_container_deploy=true"
```

NOTE	The error in TASK `[aac_compliance_db : Create compliance_reader role]` is expected and can be ignored.

Verify:

```
podman exec -it aac-postgresql psql -U compliance -d compliance -c "\dt"
```

Expect:

```
 Schema |           Name             | Type  |   Owner
--------+----------------------------+-------+------------
 public | compliance_certifications | table | compliance
 public | compliance_facts          | table | compliance
 public | compliance_results        | table | compliance
 public | customer_frameworks       | table | compliance
 public | framework_catalog         | table | compliance
 public | host_facts                | table | compliance
 public | remediation_budgets       | table | compliance
 public | remediation_rates         | table | compliance
 public | sidecar_datasets          | table | compliance
 public | technical_debt_items      | table | compliance
 public | technical_debt_summary    | table | compliance
(11 rows)
```

7.4. REGO policy loading

The README verify curls are **not** the load step.

Install the dependencies

```
ansible-galaxy collection install amazon.aws
```

```
ansible-playbook -J ansible/playbooks/load_opa_policies.yml \
```

```
  -e "opa_security_url=http://compliance.home.lab:8181" \
  -e "opa_compliance_url=http://compliance.home.lab:8182" \
  -e "opa_ot_url=http://compliance.home.lab:8183"
```

Verify:

```
curl -s http://compliance.home.lab:8181/v1/policies | jq '.result | length'
curl -s http://compliance.home.lab:8182/v1/policies | jq '.result | length'
curl -s http://compliance.home.lab:8183/v1/policies | jq '.result | length'
```

Combined expected: on the order of 460+ policies (varies by submodule revision)

Empty `{"result":[]}` after deploy is normal **until** this step runs.

7.5. Grafana installation

Install Grafana

```
podman volume create compliance_grafanadata
podman run -d --name aac-grafana \
  -p 3000:3000 \
  --restart=always \
  -v compliance_grafanadata:/var/lib/grafana \
  -e GF_SECURITY_ADMIN_USER=admin \
  --secret grafana_admin_password,type=env,target=GF_SECURITY_ADMIN_PASSWORD \
  -e GF_SERVER_HTTP_ADDR=0.0.0.0 \
  -e GF_SERVER_PORT=3000 \
  -e GF_SERVER_ROOT_URL=http://compliance.home.lab:3000 \
  -e GF_INSTALL_PLUGINS=grafana-postgresql-datasource \
  registry.redhat.io/rhel9/grafana:latest
```

TIP `GF_SECURITY_ADMIN_PASSWORD` only takes effect on first-time initialisation against an empty database. If you re-create the container and authentication fails, delete the volume first: `podman volume rm compliance_grafanadata && podman volume create compliance_grafanadata`.

Verify:

```
curl -s http://compliance.home.lab:3000/api/health
```

NOTE `ansible/playbooks/deploy_grafana_dashboard.yml` does **not** start Grafana. It only configures the datasource + imports dashboards against an **existing** Grafana on `:3000`.

```
ansible-playbook -J ansible/playbooks/deploy_grafana_dashboard.yml \
  -e "grafana_url=http://compliance.home.lab:3000" \
  -e "pg_host=compliance.home.lab"
```

7.6. Firewall configuration

Ports you can skip (not running):

Port	Purpose
5000–5004	EDA webhooks
8004	MCP

NOTE Do not open 5432 on the firewall if AAP can reach Postgres on a private interface

```
# for port in 3000 5000 5001 5002 5003 5004 5432 8004 8085 8086 8087 8181 8182 8183;
do
for port in 3000 5432 8181 8182 8183; do
  sudo firewall-cmd --permanent --add-port=${port}/tcp
done
sudo firewall-cmd --reload
```

Verify:

```
sudo firewall-cmd --list-ports
```

7.7. Quadlets

Turn off the broken system units (as root once only)

```
sudo systemctl disable --now container-opa-security.service \
  container-opa-compliance.service container-opa-ot.service

sudo rm -f /etc/systemd/system/container-opa-*.service \
  /etc/systemd/system/container-aac-postgresql.service \
  /etc/systemd/system/container-aac-grafana.service

sudo systemctl daemon-reload
```

As compliance — generate Quadlets for containers

```
mkdir -p ~/.config/containers/systemd/
cd ~/.config/containers/systemd/
```

Copy the files in the quadlets directory to the systemd directory

Remove the hand-started containers so Quadlet owns them

```
podman rm -f opa-security opa-compliance opa-ot aac-postgresql aac-grafana
```

Enable on the user bus (no sudo)

```
systemctl --user daemon-reload
systemctl --user list-unit-files | grep -E 'opa|aac'
systemctl --user start \
  opa-security.service \
  opa-compliance.service \
  opa-ot.service \
  aac-postgresql.service \
  aac-grafana.service
systemctl --user is-active \
  opa-security.service \
  opa-compliance.service \
  opa-ot.service \
  aac-postgresql.service \
  aac-grafana.service
```

Survive logout/reboot without an interactive login

```
loginctl enable-linger compliance
loginctl show-user compliance -p Linger
systemctl --user --failed
```

8. AAP Integration

8.1. Project Sync

1. AAP UI → **Projects** → **Add**
2. Source Control URL: your compliance fork/clone
3. Credential: GitHub PAT (Source Control)
4. Enable update on launch / sync options as required
5. Ensure submodules initialize (`policies/`)

8.2. Build and push the Compliance EE

```
cd install/existing-prod/ee
```

```
podman login registry.redhat.io
ansible-builder build -v -f ee/execution-environment.yml --container-runtime podman \
  -t quay.io/compliance/aac-compliance-ee:latest
podman login quay.io
podman push quay.io/compliance/aac-compliance-ee:latest
```

8.3. Launch the Compliance assessment

Launch Job template: `ansible/playbooks/cis_rhel9_assessment.yml` from AAP UI. Expect `policy_name` like `CIS RHEL 9` in Grafana **AAC Infrastructure Compliance**.